“Proyecto Alohandes: Iteración 4”

Lina M. Ojeda Amaya, Juan S. Urrea López
Universidad de los Andes, Bogotá, Colombia
{l.ojedaa, js.urrea}@uniandes.edu.co
Fecha de presentación: Mayo 28 de 2023

Tabla de contenido

1 Introducción 1
2 Analisis y diseño de la aplicación 2
3 Construccion de la aplicación 2
3.1 Ajuste y modificación de tablas **2.1**
3.2 Implementación de los requerimientos **2.2**
4 Bibliografía **3**

1 Introducción

Este trabajo se basa en la tercera iteración de la implementación de un sistema de reservas de alojamiento denominado AlohAndes, el cual ha sido desarrollado previamente en dos iteraciones anteriores. El objetivo principal de este sistema es permitir a los usuarios realizar reservas en diferentes tipos de alojamiento, tales como viviendas universitarias, apartamento amoblado, vivienda compartida etc. Esto en distintas ubicaciones y por distintas cantidades de tiempo.

En esta tercera iteración, se ha incorporado nuevos requerimientos de consulta y operaciones de modificación masiva. Se ha realizado un código SQL de población de tablas sobre las tablas creadas en la segunda iteración además se ha modificado el modelo conceptual presentado en las iteraciones pasadas esto para poder desarrollar de mejor manera la consulta de datos masiva.

En este trabajo se presentará el diseño e implementación de esta funcionalidad en el sistema AlohAndes. Se explicará el modelo de datos utilizado, así como la arquitectura de la aplicación y las tecnologías utilizadas. También se presentará una serie de consultas SQL que permitirán obtener información relevante para la toma de decisiones en el negocio, como las fechas de mayor demanda, las de mayores ingresos y las de menor ocupación para un tipo de alojamiento y una unidad de tiempo definida entre otros.

En resumen, este trabajo se enfoca en la mejora del sistema de reservas de alojamiento AlohAndes que se ha venido desarrollando por iteraciones a lo largo del semestre, esto con el fin de brindar un servicio más especializado a ciertos tipos de clientes y ofrecer información valiosa para la toma de decisiones empresariales.

2 Análisis y diseño de la aplicación.

Esta iteración ha requerido el ajuste del modelo conceptual presentado en las iteraciones anteriores, lo que implicó una inserción de tabla adicional denominada Reservas colectivas y es

acá donde se lleva el registro de las reservas que se crearon atreves de una reserva compartida. A continuación se muestra el modelo conceptual actualizado.

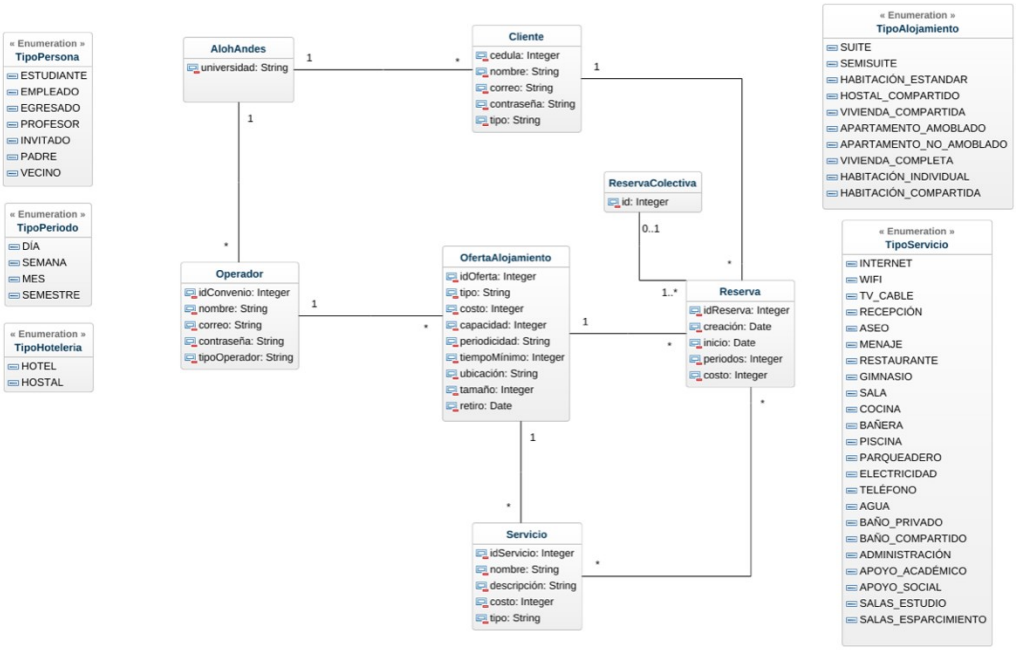

Figura 1. Modelo conceptual Actualizado

Ahora bien cabe recordar que el modelo de datos del sistema AlohAndes se encuentra en BCNF porque todas sus tablas cumplen con que cada atributo de la tabla depende únicamente de la llave primaria de la misma. Esto evita la redundancia de información y facilita la actualización y asegura la integridad de datos almacenados en nuestra base de datos.

3 Diseño físico

3.1 Índices

1. Índices propios

Luego de realizar el análisis de la aplicación y del diseño inicial se decidido que para tener una mejor eficiencia en los requerimientos de consulta era necesario agregar indicies a la aplicación y se agregaron un total de 6 índices como se muestra en la imagen 1. Se crearon índices tipo hash y B+ que se describirán a continuación:

Índices hash:

Para diferentes requerimientos nos dimos cuenta que era necesario tener una búsqueda rápida basada en un valor único, por lo que para requerimientos de búsqueda de igualdad exacta es eficiente además el costo era mínimo ya que la implementación de este tipo de índices tiene un costo pequeño ya que solo requiere guardar claves y ubicaciones físicas específicas. Por lo explicado anteriormente se crearon los índices:

- OfertaAlojamiento por operador
- OfertaAlojamiento por tipo
- Servicio por oferta
- Reserva por oferta

- Reserva por cliente

Índices B+:

Para diferentes requerimientos se requería buscar en un rango de fechas en específico por lo que la creación de un índice tipo árbol nos serviría para buscar por rango de valores un criterio en específico. Aunque n cuanto a costo de almacenamiento este si tiene un costo importante se compensa con el rendimiento eficiente en las operaciones de búsqueda y recorrido en orden. Por lo explicado anteriormente se crearon los índices:

- Reserva(Inicio)

```
-- Indexes
CREATE INDEX FK_OfertaAlojamiento_Operador on OfertaAlojamiento(Operador);
CREATE INDEX FK_OfertaAlojamiento_Tipo on OfertaAlojamiento(Tipo);
CREATE INDEX FK_Reserva_Oferta on Reserva(Oferta);
CREATE INDEX FK_Reserva_Cliente on Reserva(Cliente);
CREATE INDEX FK_Reserva_Inicio on Reserva(Inicio);
CREATE INDEX FK_Servicio_Oferta on Servicio(Oferta);
```

Imagen 1. Creación de índices

2. Índices creados de forma automática por Oracle

```
select * from dba_indexes where table_owner = 'ISIS2304D33202310';
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 10 en 0,557 segundos

	OWNER	INDEX_NAME	INDEX_TYPE	TABLE_OWNER	TABLE_NAME	TABLE_TYPE	UNIQUENESS	COMPRESSION	PREFIX_LENGTH	TABLESPACE_NAME	INI_T
1	ISIS2304D33202310	SYS_C001058312	NORMAL	ISIS2304D33202310	ALOHANDES	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
2	ISIS2304D33202310	SYS_C001058320	NORMAL	ISIS2304D33202310	CLIENTE	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
3	ISIS2304D33202310	SYS_C001058321	NORMAL	ISIS2304D33202310	CLIENTE	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
4	ISIS2304D33202310	SYS_C001058345	NORMAL	ISIS2304D33202310	OFERTAALOJAMIENTO	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
5	ISIS2304D33202310	SYS_C001058363	NORMAL	ISIS2304D33202310	OPERADOR	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
6	ISIS2304D33202310	SYS_C001058364	NORMAL	ISIS2304D33202310	OPERADOR	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
7	ISIS2304D33202310	SYS_C001058331	NORMAL	ISIS2304D33202310	RESERVA	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
8	ISIS2304D33202310	PK_RESERVACOLECTIVA	NORMAL	ISIS2304D33202310	RESERVACOLECTIVA	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
9	ISIS2304D33202310	SYS_C001058352	NORMAL	ISIS2304D33202310	SERVICIO	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	
10	ISIS2304D33202310	PK_SERVICIOADQUIRIDO	NORMAL	ISIS2304D33202310	SERVICIOADQUIRIDO	TABLE	UNIQUE	DISABLED	(null)	TBSPROD	

3.2 Análisis

En esta iteración se terminó de construir la aplicación completando los requerimientos de consulta RFC10, RFC11, RFC12 y RFC13 A continuación se muestran todas las consultas SQL incluyendo las de las anteriores iteraciones con sus respectivas pruebas de igual manera anexo a este documento se encontrara el txt con las consultas independientes.

1. Ajustes y población de tablas

- Todas las tablas implementadas y creadas en la iteración anterior se mantuvieron y fueron pobladas para la generación de pruebas en esta iteración. El código de la población de tablas se encuentra en el SQL adjunto en esta iteración.

- Se genero el archivo de más de un millón de datos donde se pudo verificar la eficiencia de la aplicación.

Para la población de tablas y realización de pruebas se usaron de base las siguientes tablas:

Clientes					
Cedula	Nombre	Correo	contrasena	tipo	alohandes
1001	JUAN A	juan@a.com	password	ESTUDIANTE	Universidad de los andes
1002	MARIA A	maria@a.com	password	ESTUDIANTE	Universidad de los andes
1003	JUAN B	juan@b.com	password	ESTUDIANTE	Universidad de los andes
1004	MARIA B	maria@b.com	password	ESTUDIANTE	Universidad de los andes
1005	JUAN C	juan@c.com	password	ESTUDIANTE	Universidad de los andes
1006	MARIA C	maria@c.com	password	PROFESOR	Universidad de los andes
1007	JUAN D	juan@d.com	password	PROFESOR	Universidad de los andes
1008	MARIA D	maria@d.com	password	EGRESADO	Universidad de los andes
1009	JUAN E	juan@e.com	password	EGRESADO	Universidad de los andes
1010	MARIA E	maria@e.com	password	EMPLEADO	Universidad de los andes

Tabla 1. Tabla de pruebas clientes

operador					
id_convenio	nombre	correo	contrasena	tipo_operador	alohandes
1	CityA	city@a.com	password	RECIDENCIA_UNIVERSITARIA	Universidad de los andes
2	CityB	city@b.com	password	RECIDENCIA_UNIVERSITARIA	Universidad de los andes
3	CityC	city@c.com	password	RECIDENCIA_UNIVERSITARIA	Universidad de los andes
4	CityD	city@d.com	password	RECIDENCIA_UNIVERSITARIA	Universidad de los andes
5	CityE	city@e.com	password	RECIDENCIA_UNIVERSITARIA	Universidad de los andes
6	APTO101	apto@101.com	password	APARTAMENTO_NO_AMOBLADO	Universidad de los andes
7	APTO102	apto@102.com	password	APARTAMENTO_AMOBLADO	Universidad de los andes
8	APTO103	apto@103.com	password	APARTAMENTO_NO_AMOBLADO	Universidad de los andes
9	APTO104	apto@104.com	password	APARTAMENTO_AMOBLADO	Universidad de los andes
10	APTO105	apto@105.com	password	APARTAMENTO_NO_AMOBLADO	Universidad de los andes

Tabla 2. Tabla de pruebas operador

ofertaalojamiento									
id_oferta	tipo	costo	capacidad	periocidad	tiempo_minimo	ubicación	tamano	retiro	operador
1	HABITACION_COMPARTIDA	25	5	SEMANA	1	GERMANIA	50	(null)	1
2	HABITACION_INDIVIDUAL	50	5	SEMANA	1	GERMANIA	50	(null)	2
3	HABITACION_COMPARTIDA	25	5	SEMANA	1	GERMANIA	50	(null)	3
4	HABITACION_INDIVIDUAL	50	5	SEMANA	1	GERMANIA	50	(null)	2
5	HABITACION_COMPARTIDA	25	5	SEMANA	1	GERMANIA	50	(null)	5
6	APARTAMENTO_NO_AMOBLADO	150	3	MES	1	GERMANIA	40	(null)	6
7	APARTAMENTO_AMOBLADO	200	3	MES	1	GERMANIA	40	(null)	7
8	APARTAMENTO_NO_AMOBLADO	150	3	MES	1	GERMANIA	40	(null)	8
9	APARTAMENTO_AMOBLADO	200	3	MES	1	GERMANIA	40	(null)	7
10	APARTAMENTO_NO_AMOBLADO	150	3	MES	1	GERMANIA	40	(null)	10

Tabla 3. Tabla de pruebas ofertaalojamiento

reserva						
id_reserva	creacion	inicio	periodos	costo	cliente	oferta
1	7/05/23	15/05/23	1	25	1001	1
2	7/05/23	20/05/23	1	25	1001	1
3	7/05/23	22/05/23	1	25	1001	1
4	7/05/23	1/05/23	1	25	1002	1
5	7/05/23	15/05/23	3	150	1002	2
6	7/05/23	15/05/23	13	650	1002	2
7	7/05/23	15/05/23	3	75	1002	3
8	7/05/23	15/05/23	3	75	1003	3
9	7/05/23	15/05/23	3	150	1004	4
10	7/05/23	15/05/23	3	75	1005	5
11	7/05/23	15/05/23	3	450	1006	6
12	7/05/23	15/05/23	3	600	1007	7
13	7/05/23	15/05/23	3	450	1008	8
14	7/05/23	15/05/23	3	600	1009	9
15	7/05/23	20/05/23	3	600	1010	9

Tabla 4. Tabla de pruebas reserva

servicio					
id_servicio	nombre	descripcion	costo	tipo	oferta
1	CityLobby	Recepción 24 horas	5	RECEPCION	1
2	CityLobby	Recepción 24 horas	0	RECEPCION	2
3	CityLobby	Recepción 24 horas	0	RECEPCION	3
4	CityLobby	Recepción 24 horas	0	RECEPCION	4
5	CityLobby	Recepción 24 horas	0	RECEPCION	5
6	Aptolobby	Recepción 24 horas	0	RECEPCION	6
7	Aptolobby	Recepción 24 horas	0	RECEPCION	7
8	Aptolobby	Recepción 24 horas	0	RECEPCION	8
9	Aptolobby	Recepción 24 horas	0	RECEPCION	9
10	Aptolobby	Recepción 24 horas	0	RECEPCION	10

Tabla 5. Tabla de pruebas servicio

servicioadquirido	
servicio	reserva
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10

Tabla 6. Tabla de pruebas servicio adquirido

Resultados de las consultas:

- RFC1:
  - Consulta SQL:

```
SELECT OP.nombre, SUM(SQ.suma) AS INGRESOS
FROM OPERADOR OP
LEFT JOIN OFERTAALOJAMIENTO OA
ON OP.id_convenio = OA.operador
LEFT JOIN (
   SELECT R.oferta, SUM(R.costo) AS SUMA FROM RESERVA R
   WHERE R.inicio BETWEEN TRUNC(SYSDATE, 'YEAR') AND
ADD_MONTHS(TRUNC(SYSDATE, 'YEAR'), 12)
   GROUP BY R.oferta
) SQ ON OA.id_oferta = SQ.oferta
GROUP BY OP.nombre;
```

- Resultado de prueba:

	NOMBRE	INGRESOS
1	APTO102	1800
2	APTO105	(null)
3	CityI	150
4	APTO101	450
5	CityE	950
6	APTO103	450
7	APTO104	(null)
8	CityA	100
9	CityU	75
10	CityO	(null)

- Plan ejecución Oracle:

SQL 0.204 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			10	13
HASH		GROUP BY	10	13
HASH JOIN		OUTER	12	12
Access Predicates				
OA.ID_OFERTA=SQ.OFERTA(+)				
MERGE JOIN		OUTER	12	7
TABLE ACCESS	OPERADOR	BY INDEX ROWID	10	2
INDEX	SYS_C001060665	FULL SCAN	10	1
SORT		JOIN	10	5
Access Predicates				
OP.ID_CONVENIO=OA.OPERADOR(+)				
Filter Predicates				
OP.ID_CONVENIO=OA.OPERADOR(+)				
TABLE ACCESS	OFERTAALOJAMIENTO	FULL	10	4
VIEW			9	5
HASH		GROUP BY	9	5
FILTER				
Filter Predicates				
ADD_MONTHS(TRUNC(SYSDATE@!,'fmyear'),12)>=TRUNC(SYSDATE@!,'fmyear')				
TABLE ACCESS	RESERVA	FULL	15	4
Filter Predicates				
AND				
R.INICIO>=TRUNC(SYSDATE@!,'fmyear')				
R.INICIO<=ADD_MONTHS(TRUNC(SYSDATE@!,'fmyear'),12)				

- RCF2:
  - Consulta SQL:

```
SELECT * FROM (
```

```
SELECT OA.id_oferta, COUNT(R.id_reserva) AS RESERVAS FROM OFERTAALOJAMIENTO OA
  LEFT JOIN RESERVA R ON OA.id_oferta = R.oferta
  GROUP BY OA.id_oferta
  ORDER BY RESERVAS DESC
) WHERE ROWNUM <= 20;
```

- Resultado de prueba:

ID_OFERTA	RESERVAS
1	4
2	2
9	2
3	2
4	1
5	1
6	1
7	1
8	1
10	0

- Plan de ejecución Oracle:

SQL 0.038 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			9	7
COUNT		STOPKEY		
Filter Predicates				
ROWNUM<=20				
VIEW			9	7
SORT		ORDER BY STOPKEY	9	7
Filter Predicates				
ROWNUM<=20				
SORT		GROUP BY NOSORT	9	7
MERGE JOIN		OUTER	16	6
VIEW	SYS.VW_GBF_6		10	1
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		JOIN	15	5
Access Predicates				
ITEM_1=R.OFERTA(+)				
Filter Predicates				
ITEM_1=R.OFERTA(+)				
TABLE ACCESS	RESERVA	FULL	15	4

- RCF3:
  - Consulta SQL

```
SELECT OA.id_oferta, NULLIF(SQ.reservas, 0) / OA.capacidad AS INDICE
FROM OFERTAALOJAMIENTO OA
LEFT JOIN (
  SELECT R.oferta, COUNT(R.id_reserva) AS RESERVAS
  FROM RESERVA R
  GROUP BY R.oferta
) SQ ON OA.id_oferta = SQ.oferta;
```

- Resultado prueba:

	ID_OFERTA	INDICE
1	1	0,8
2	2	0,4
3	3	0,4
4	4	0,2
5	5	0,2
6	6	0,3333333333333333333333333333333333333333
7	7	0,3333333333333333333333333333333333333333
8	8	0,3333333333333333333333333333333333333333
9	9	0,6666666666666666666666666666666666666667
10	10	(null)

- Plan de ejecución Oracle

SQL 0.048 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			10	8
MERGE JOIN		OUTER	10	8
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	10	2
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		JOIN	9	6
Access Predicates				
OA.ID_OFERTA=SQ.OFERTA(+)				
Filter Predicates				
OA.ID_OFERTA=SQ.OFERTA(+)				
VIEW			9	5
HASH		GROUP BY	9	5
TABLE ACCESS	RESERVA	FULL	15	4

- RCF4:

  - Consulta SQL

SELECT CL.tipo, COUNT(R.id_reserva) AS Reservas, SUM(costo) AS Ingresos, SUM(R.periodos) AS Periodos
FROM CLIENTE CL
LEFT JOIN RESERVA R
ON R.cliente = CL.cedula
GROUP BY CL.tipo;

  - Resultado pruebas

Con tipo de servicio: Recepción

ID_OFERTA
1
2
3
4
5
6
7
8
9
10

- Plan ejecución Oracle

SQL | 0.036 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			10	8
SORT		UNIQUE NOSORT	10	8
MERGE JOIN		SEMI	10	7
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	10	2
Filter Predicates				
OA.RETIRO IS NULL				
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		UNIQUE	10	5
Access Predicates				
OA.ID_OFERTA=S.OFERTA				
Filter Predicates				
OA.ID_OFERTA=S.OFERTA				
TABLE ACCESS	SERVICIO	FULL	10	4
Filter Predicates				
S.TIPO='RECEPCION'				

- RCF5:

  - Consulta SQL

```
SELECT CL.tipo, COUNT(R.id_reserva) AS Reservas, SUM(costo) AS Ingresos,
SUM(R.periodos) AS Periodos
FROM CLIENTE CL
LEFT JOIN RESERVA R
ON R.cliente = CL.cedula
GROUP BY CL.tipo;
```

  - Resultado Pruebas

	TIPO	RESERVAS	INGRESOS	PERIODOS
1	EMPLEADO	1	600	3
2	ESTUDIANTE	9	1200	29
3	PROFESOR	3	1125	9
4	EGRESADO	2	1050	6

  - Plan de ejecución Oracle

SQL | 0.061 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			4	8
HASH		GROUP BY	4	8
MERGE JOIN		OUTER	15	7
TABLE ACCESS	CLIENTE	BY INDEX ROWID	10	2
INDEX	SYS_C001060622	FULL SCAN	10	1
SORT		JOIN	15	5
Access Predicates				
R.CLIENTE(+)=CL.CEDULA				
Filter Predicates				
R.CLIENTE(+)=CL.CEDULA				
TABLE ACCESS	RESERVA	FULL	15	4

- RCF6:

  - Consulta SQL

```
SELECT OP.nombre, OP.tipo_operador, OA.tipo, OA.ubicacion, OA.periodicidad,
R.periodos, R.costo, R.inicio
FROM RESERVA R
LEFT JOIN OFERTAALOJAMIENTO OA
ON R.oferta = OA.id_oferta
LEFT JOIN OPERADOR OP
ON OA.operador = OP.id_convenio
WHERE R.cliente = 1001
ORDER BY R.creacion DESC;
```

- Resultado prueba:

Con cliente: 1001

	NOMBRE	TIPO_OPERADOR	TIPO	UBICACION	PERIODICIDAD	PERIODOS	COSTO	INICIO
1	CityA	RESIDENCIA_UNIVERSITARIA	HABITACION_COMPARTIDA	GERMANIA	SEMANA	1	25	15/05/23
2	CityA	RESIDENCIA_UNIVERSITARIA	HABITACION_COMPARTIDA	GERMANIA	SEMANA	1	25	22/05/23
3	CityA	RESIDENCIA_UNIVERSITARIA	HABITACION_COMPARTIDA	GERMANIA	SEMANA	1	25	20/05/23

- Plan ejecucion Oracle

SQL | 0.084 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			2	9
SORT		ORDER BY	2	9
HASH JOIN		OUTER	2	8
Access Predicates				
OA.OPERADOR=OP.ID_CONVENIO(+)				
NESTED LOOPS		OUTER	2	8
STATISTICS COLLECTOR				
HASH JOIN		OUTER	2	6
Access Predicates				
R.OFERTA=OA.ID_OFERTA(+)				
NESTED LOOPS		OUTER	2	6
STATISTICS COLL				
TABLE ACCESS	RESERVA	FULL	2	4
Filter Predicates				
R.CLIENTE=1001				
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	1	1
INDEX	SYS_C001060647	UNIQUE SCAN	1	0
Access Predicates				
R.OFERTA=OA.ID_OFERTA(+)				
TABLE ACCESS	OFERTAALOJAMIENTO	FULL	1	1
TABLE ACCESS	OPERADOR	BY INDEX ROWID	1	1
INDEX	SYS_C001060665	UNIQUE SCAN	1	0
Access Predicates				
OA.OPERADOR=OP.ID_CONVENIO(+)				
TABLE ACCESS	OPERADOR	FULL	1	1

- RCF7:

Consulta SQL:

```
SELECT TO_CHAR(R.inicio, 'DD-MM-YY') AS fecha, COUNT(*) AS Reservas,
SUM(R.costo) AS Ingresos, SUM(OA.capacidad) AS Capacidad
FROM RESERVA R
LEFT JOIN OFERTAALOJAMIENTO OA
ON OA.id_oferta = R.oferta
WHERE OA.tipo = 'HABITACION_INDIVIDUAL'
```

GROUP BY TO_CHAR(R.inicio, 'DD-MM-YY');

- Resultados prueba:

Con tipo de alojamiento: Habitación individual
Con semana en especifico:

	FECHA	RESERVAS	INGRESOS	CAPACIDAD
1	15-05-23	3	950	15

- Plan ejecución Oracle:
-

SQL | 0.069 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			4	8
HASH		GROUP BY	4	8
MERGE JOIN			4	7
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	3	2
Filter Predicates				
OA.TIPO='HABITACION_INDIVIDUAL'				
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		JOIN	15	5
Access Predicates				
OA.ID_OFERTA=R.OFERTA				
Filter Predicates				
OA.ID_OFERTA=R.OFERTA				
TABLE ACCESS	RESERVA	FULL	15	4

- RCF8:

  - Consulta SQL:

    SELECT c.nombre, c.cedula, COUNT(*) as utilizado, SUM(r.periodos) as noches
    FROM cliente c
    INNER JOIN reserva r ON c.cedula = r.cliente
    WHERE r.oferta = 1
    GROUP BY c.nombre, c.cedula
    HAVING COUNT(*) >= 3 OR SUM(r.periodos) >= 15;

  - Resultado prueba:

	NOMBRE	CEDULA	UTILIZADO	NOCHES
1	JUAN A	1001	3	3

  - Plan ejecucion Oracle:

SQL | 0.057 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			1	8
FILTER				
Filter Predicates				
OR				
COUNT(*)>=3				
SUM(R.PERIODOS)>=15				
HASH		GROUP BY	1	8
MERGE JOIN			4	7
TABLE ACCESS	CLIENTE	BY INDEX ROWID	10	2
INDEX	SYS_C001060622	FULL SCAN	10	1
SORT		JOIN	4	5
Access Predicates				
C.CEDULA=R.CLIENTE				
Filter Predicates				
C.CEDULA=R.CLIENTE				
TABLE ACCESS	RESERVA	FULL	4	4
Filter Predicates				
R.OFERTA=1				

- RCF9:

  - Consulta SQL:

```
SELECT ofe.id_oferta, ofe.tipo
FROM ofertaalojamiento ofe
WHERE NOT EXISTS (
  SELECT 1
  FROM reserva r
  WHERE r.oferta = ofe.id_oferta
    AND r.inicio >= (SYSDATE - INTERVAL '1' MONTH));
```

  - Resultado prueba:

	ID_OFERTA	TIPO
1	10	APARTAMENTO_NO_AMOBLADO

  - Plan ejecucion Oracle:

SQL | 0.022 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			1	7
MERGE JOIN		ANTI	1	7
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	10	2
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		UNIQUE	15	5
Access Predicates				
R.OFERTA=OFE.ID_OFERTA				
Filter Predicates				
R.OFERTA=OFE.ID_OFERTA				
TABLE ACCESS	RESERVA	FULL	15	4
Filter Predicates				
R.INICIO>=SYSDATE@!-INTERVAL'+00-01' YEAR(2) TO MONTH				

RCF10:

RCF11:

- RCF12:

  - Consulta SQL:

```
WITH ocupacion_semanal AS (
   SELECT
      TO_CHAR(reserva.inicio, 'WW') AS semana,
      reserva.oferta,
      COUNT(*) AS cantidad_reservas
   FROM
      reserva
   GROUP BY
      TO_CHAR(reserva.inicio, 'WW'),
      reserva.oferta
),
operadores_solicitados AS (
   SELECT
      TO_CHAR(reserva.inicio, 'WW') AS semana,
      ofertaalojamiento.operador,
      COUNT(*) AS cantidad_reservas
   FROM
      reserva
      INNER JOIN ofertaalojamiento ON reserva.oferta = ofertaalojamiento.id_oferta
   GROUP BY
      TO_CHAR(reserva.inicio, 'WW'),
      ofertaalojamiento.operador
)
SELECT
   oc.semana,
   MAX(a.id_oferta) AS oferta_mas_ocupada,
   MIN(a.id_oferta) AS oferta_menos_ocupada,
   MAX(op.operador) AS operador_mas_solicitado,
   MIN(op.operador) AS operador_menos_solicitado
FROM
   ocupacion_semanal oc
   INNER JOIN ofertaalojamiento a ON oc.oferta = a.id_oferta
   INNER JOIN operadores_solicitados op ON oc.semana = op.semana AND a.operador =
op.operador
GROUP BY
   oc.semana;
```

- Resultado pruebas:

	SEMANA	OFERTA_MAS_OCUPADA	OFERTA_MENOS_OCUPADA	OPERADOR_MAS_SOLICITADO	OPERADOR_MENOS_SOLICITADO
1	20	9	1	8	1
2	18	1	1	1	1
3	21	1	1	1	1

- Plan ejecucion Oracle:

SQL | 0.067 segundos

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
HASH		GROUP BY	1	16
HASH JOIN		SEMI	1	15
Access Predicates				
AND				
A.OPERADOR=OP.OPERADOR				
ITEM_1=A.ID_OFERTA				
NESTED LOOPS		SEMI	1	15
STATISTICS COLLECTOR				
HASH JOIN			1	14
Access Predicates				
ITEM_2=OP.SEMANA				
VIEW	SYS.VW_GBF_5		9	6
HASH		GROUP BY	9	6
VIEW			15	5
HASH		GROUP BY	15	5
TABLE A(	RESERVA	FULL	15	4
VIEW			15	8
HASH		GROUP BY	15	8
MERGE JOIN			15	7
TABLE ACC	OFERTAALOJAMIENTO	BY INDEX ROWID	10	2
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		JOIN	15	5
Access Predicates				
RESERVA.OFERTA=OFERTAALOJAMIENTO.ID_OFERTA				
Filter Predicates				
RESERVA.OFERTA=OFERTAALOJAMIENTO.ID_OFERTA				
TABLE A(	RESERVA	FULL	15	4
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	10	1
Filter Predicates				
A.OPERADOR=OP.OPERADOR				
INDEX	SYS_C001060647	UNIQUE SCAN	1	0
Access Predicates				
ITEM_1=A.ID_OFERTA				
TABLE ACCESS	OFERTAALOJAMIENTO	FULL	10	1

- RCF13:
    - Consulta SQL:

```
SELECT
   c.cedula,
   c.nombre,
   c.correo,
   (
      SELECT COUNT(*)
      FROM reserva r
      WHERE r.cliente = c.cedula
      AND r.inicio >= ADD_MONTHS(TRUNC(SYSDATE, 'MM'), -1)
   ) AS reservas_mensuales,
   (
      SELECT COUNT(*)
      FROM reserva r
      INNER JOIN ofertaalojamiento a ON r.oferta = a.id_oferta
      WHERE r.cliente = c.cedula
      AND a.costo > 150
   ) AS reservas_costosas,
   (
      SELECT COUNT(*)
      FROM reserva r
```

```
        INNER JOIN ofertaalojamiento a ON r.oferta = a.id_oferta
        WHERE r.cliente = c.cedula
        AND a.tipo = 'SUITE'
    ) AS reservas_suites
FROM
    cliente c
WHERE
    (
        EXISTS (
            SELECT 1
            FROM reserva r
            WHERE r.cliente = c.cedula
            AND r.inicio >= ADD_MONTHS(TRUNC(SYSDATE, 'MM'), -1)
        )
        OR EXISTS (
            SELECT 1
            FROM reserva r
            INNER JOIN ofertaalojamiento a ON r.oferta = a.id_oferta
            WHERE r.cliente = c.cedula
            AND a.costo > 150
        )
        OR EXISTS (
            SELECT 1
            FROM reserva r
            INNER JOIN ofertaalojamiento a ON r.oferta = a.id_oferta
            WHERE r.cliente = c.cedula
            AND a.tipo = 'SUITE'
        )
    );
```

	CEDULA	NOMBRE	CORREO	RESERVAS_MENSUALES	RESERVAS_COSTOSAS	RESERVAS_SUITES
1	1001	JUAN A	juan@a.com	3	0	0
2	1002	MARIA A	maria@a.com	3	0	0
3	1003	JUAN B	juan@b.com	1	0	0
4	1004	MARIA B	maria@b.com	1	0	0
5	1005	JUAN C	juan@c.com	1	0	0
6	1006	MARIA C	maria@c.com	2	0	0
7	1007	JUAN D	juan@d.com	1	1	0
8	1008	MARIA D	maria@d.com	1	0	0
9	1009	JUAN E	juan@e.com	1	1	0
10	1010	MARIA E	maria@e.com	1	1	0

Plan ejecucion oracle:

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT			1	29
SORT		AGGREGATE	1	
TABLE ACCESS	RESERVA	FULL	2	4
Filter Predicates				
AND				
R.CLIENTE=:B1				
R.INICIO>=ADD_MONTHS(TRUNC(SYSDATE@!,'fmmm'),(-1))				
SORT		AGGREGATE	1	
NESTED LOOPS			1	6
NESTED LOOPS			2	6
TABLE ACCESS	RESERVA	FULL	2	4
Filter Predicates				
R.CLIENTE=:B1				
INDEX	SYS_C001060647	UNIQUE SCAN	1	0
Access Predicates				
R.OFERTA=A.ID_OFERTA				
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	1	1
Filter Predicates				
A.COSTO>150				
SORT		AGGREGATE	1	
NESTED LOOPS			1	6
NESTED LOOPS			2	6
TABLE ACCESS	RESERVA	FULL	2	4
Filter Predicates				
R.CLIENTE=:B1				
INDEX	SYS_C001060647	UNIQUE SCAN	1	0
Access Predicates				
R.OFERTA=A.ID_OFERTA				
TABLE ACCESS	OFERTAALOJAMIENTO	BY INDEX ROWID	1	1
Filter Predicates				
A.TIPO='SUITE'				

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
HASH JOIN			1	13
Access Predicates				
VW_COL_1=C.CEDULA				
NESTED LOOPS			1	13
NESTED LOOPS			1	13
STATISTICS COLLECTOR				
VIEW	SYS.VW_SQ_1		20	11
HASH		UNIQUE	1	11
UNION-ALL				
MERGE JOIN			5	7
TABLE ACC	OFERTAALOJAMIENTO	BY INDEX ROWID	3	2
Filter Predicates				
OR				
A.COSTO>150				
A.TIPO='SUITE'				
INDEX	SYS_C001060647	FULL SCAN	10	1
SORT		JOIN	15	5
Access Predicates				
R.OFERTA=A.ID_OFERTA				
Filter Predicates				
R.OFERTA=A.ID_OFERTA				
TABLE A	RESERVA	FULL	15	4
TABLE ACCESS	RESERVA	FULL	15	4
Filter Predicates				
R.INICIO>=ADD_MONTHS(TRUNC(SYSDATE@!,'fmmm'),(-1))				
INDEX	SYS_C001060622	UNIQUE SCAN	1	0
Access Predicates				
VW_COL_1=C.CEDULA				
TABLE ACCESS	CLIENTE	BY INDEX ROWID	1	1
TABLE ACCESS	CLIENTE	FULL	1	1